exp10 — GPU shards with a per-node elastic core split and corrected Slurm scheduling for DeepPrep on the C2 cluster

DS-I (29 subjects, sMRI+fMRI rest), C2 cluster (Node4: 16 cores; Node2: 16 cores + T4 15 GB; Node3: 16 cores + 2× RTX 6000 23 GB), Slurm 24.11.7. exp10 run 2026-10-14 16:00. Baselines: exp7 (Slurm 19.05, elastic cap + remaining-critical-path priority, 137.1 min) and exp9 (first sharded run, 138.1 min).

1. What changed in exp10 (relative to exp9)

- **Per-node elastic spill cap** (partitions cpu-spill2 / cpu-spill3, stats/spill_controller2.sh, 5-s tick): cap_n = 16 − cores held by running GPU tasks on node n − 4 cores reserved only while a GPU task is pending and node n has a free shard. CPU tasks get every core the GPU tasks are not using; a GPU task that could start always finds cores. exp9 used one cap of 8 for both nodes whenever any GPU task was pending, which idled 23–25 cores for 20 min.
- **Slurm scheduler settings** — all three were 24.11 defaults that differed from 19.05 and were found during exp9/exp10: `SchedulerParameters=bf_interval=5` (was 30 s: small jobs waited behind a 6-core anat_sphere while cores idled); `PriorityType=priority/basic` (24.11 defaults to multifactor, which with no weights gives every job priority 1 and ignores `--nice`, so exp9 ran as FIFO); gpu-node `PriorityTier=2` removed (it held GPU-node cores for GPU jobs that were only waiting for a shard).
- **bold_synthmorph_joint below the anat GPU steps** (nice 2000 instead of 773). It needs an exclusive GPU (17 GB); once --nice worked again it outranked the shard tasks, so the 29 synthmorph jobs monopolized all three GPUs serially and every anatomical chain stalled (first exp10 attempt: cluster at 9 allocated cores for 7 min; aborted and relaunched).
- Unchanged from exp9: 2 shards per GPU (--gres=shard:1 for segment / levelset / sphere_register), bold_pre_process and bold_transform_chain cpus 2, exp7 priority tables, retries, container, inputs.

2. Result

Metric	exp7	exp9	exp10
Makespan (first submit → last complete)	**137.1 min**	**138.1 min** (+0.8% vs exp7)	**120.5 min** (-12.1% vs exp7)
Tasks ok / retried	2323 / 1	2323 / 0	2323 / 0
CPU work (Σ %cpu × time)	4668 core-min	4757 core-min	4682 core-min
Cluster CPU utilization	71.0%	71.7%	81.0%
Avg cores allocated (peak)	40.7 (56)	39.8 (54)	44.8 (56)
Anat GPU convoy (segment + levelset) ends at	42 min	34 min	24 min
Convoy phase: cores allocated / used	29.9 / 26.0	28.4 / 23.3	40.8 / 35.1
Last 25 min: cores allocated / used	46.2 / 31.9	34.7 / 31.2	45.9 / 40.5
GPU tasks: avg / max wait	484 / 1118 s	129 / 404 s	182 / 1168 s
GPU task run time sum	144 min	168 min	163 min
GPU busy share (T4 / RTX#0 / RTX#1)	22% / 20% / 17%	18% / 19% / 15%	21% / 22% / 18%
Peak VRAM (T4 / RTX#0 / RTX#1)	14.1 / 16.8 / 16.8 GB	14.1 / 16.8 / 16.8 GB	14.1 / 16.8 / 16.8 GB
bold_pre_process / transform_chain mean time	278 / 71 s	318 / 68 s	319 / 76 s
Queue wait median / max (all tasks)	22 / 3184 s	128 / 3178 s	10 / 3890 s
Node CPU busy, dstat (Node4 / Node2 / Node3)	78 / 67 / 73%	82 / 67 / 71%	83 / 81 / 83%
First / last subject completed	135 / 137 min	117 / 138 min	119 / 120 min

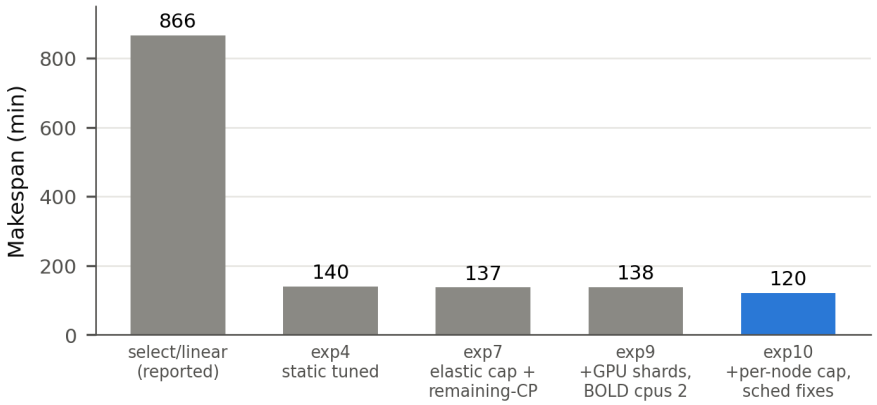


Fig. 1 Makespan for DS-I across configurations.

3. Utilization during exp10



Fig. 2 exp10: (a) per-node CPU utilization; (b) per-GPU utilization; (c) cores allocated vs actually used by running tasks (Nextflow trace) and the per-node spill caps set by the controller (cores available to CPU tasks on each GPU node).

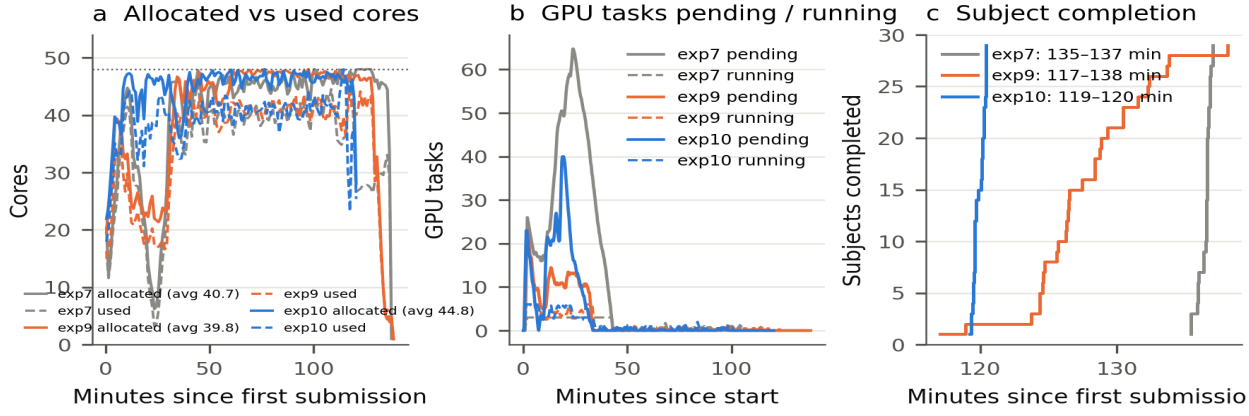


Fig. 3 exp7 vs exp9 vs exp10: (a) allocated and used cores; (b) GPU tasks pending and running; (c) subject completion.

4. Where the time went

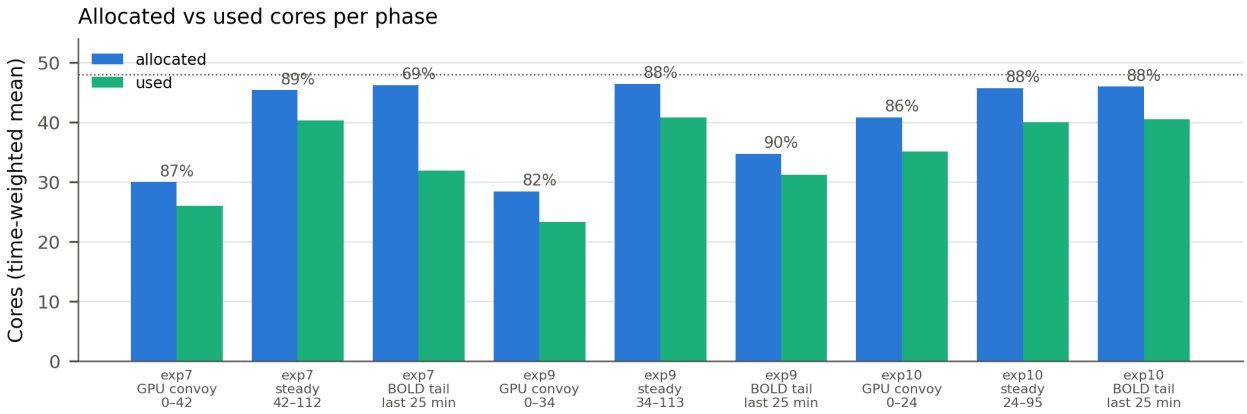


Fig. 4 Allocated vs used cores in the three phases of each run (labels: used/allocated).

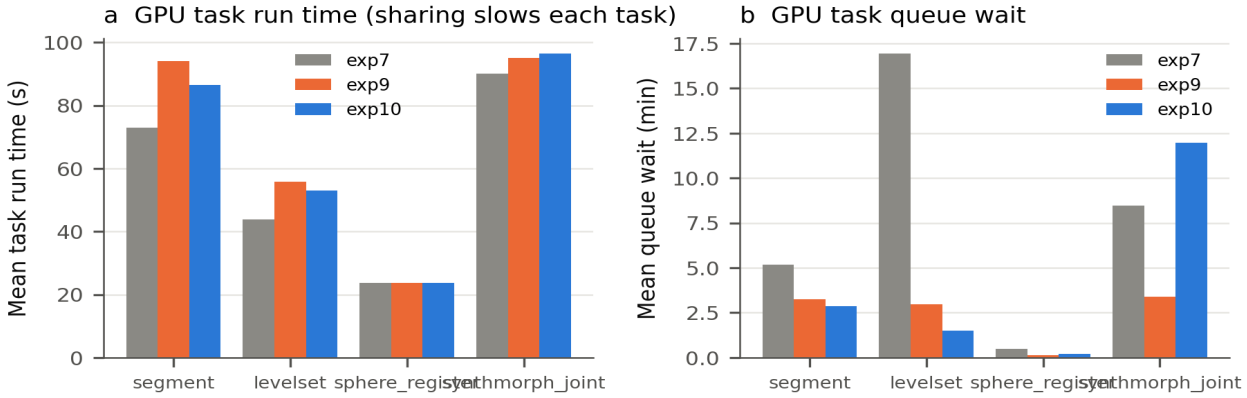


Fig. 5 GPU tasks: (a) mean run time per task type (two tasks per GPU time-share the SM); (b) mean queue wait.

5. Reading the result

Outcome: 120.5 min, 12.1% below exp7's 137.1 min and the first result under the "throughput ceiling" measured on DS-I; 2323/2323 tasks, 0 retries, identical outputs. Cluster CPU utilization rose from 71% to 81% and the average allocation from 40.7 to 44.8 of 48 cores, with the same CPU work (4682 vs 4668 core-min) — the gain is idle time removed, not work removed. The subject tail is flat again (first 119 min, last 120 min), as in exp7, because --nice is effective again.

Where the 17 min came from. (1) The anatomical GPU convoy (segment + levelset) ended at 24 min instead of 42 (exp7) or 34 (exp9): two shards per GPU plus the correct ordering (anat GPU steps before the exclusive synthmorph jobs) kept 4–5 GPU tasks running at once. (2) The CPU work this released was consumed immediately: between minute 10 and 30, idle cores while CPU tasks were pending fell from 23–25 (exp9, single cap of 8) to 3–4 (per-node caps), and allocation reached 44–45 cores by minute 10 instead of minute 40. (3) In the steady phase 5-s backfill kept idle-while-pending at 1.1–1.6 cores (exp7 had no per-node figure; exp9 before the fix 3.3). (4) The tail used 40.5 of 45.9 allocated cores (88%) versus 31.9 of 46.2 (69%) in exp7: with the BOLD steps reserving 2 cores and more subjects reaching BOLD at the same time, the last 25 min are no longer dominated by half-used 4-core reservations.

Costs and side effects. GPU sharing still slows each GPU task (segment 86 s vs 73, levelset 53 vs 44; summed GPU task time 163 vs 144 min) — acceptable because the GPU is not the bottleneck once the convoy is short. bold_synthmorph_joint now waits 12 min on average (max 19 min) because it is deliberately ranked below the anat GPU steps; it is off the critical path (its consumers are the BOLD chains, which start after the anatomical chain), so this costs nothing in makespan. bold_pre_process at 2 cores takes 319 s instead of 278 s; with the tail now dependency-bound this did not matter, but on long-BOLD datasets (DS-V, DS-VI) the trade should be re-checked.

What the 24.11 upgrade taught. Three defaults silently changed the experiment and had to be found by watching the queue: priority/multifactor with no weights flattens all priorities (nice ignored); PriorityTier reserves cores for jobs that cannot run yet; a 30-s backfill interval is too slow for 1–7-min tasks. The current slurm.conf fixes all three and is the configuration the other datasets should be re-run with. Two shards per GPU is memory-safe for every DeepPrep GPU step except synthmorph (17 GB), which keeps an exclusive GPU; three shards would fit the 2–3 GB levelset/sphere steps on the RTX 6000s but not two anat_segment plus one more, so a memory-aware shard count per task type would be the next step.

Next. Re-run DS-II…DS-VI with this configuration (exp10 config + per-node controller) to confirm the gain transfers; the scaling model predicts ~4.0 min per subject instead of 4.6. Remaining levers on DS-I are small: the first 10 min (allocation 36 of 48 while the first GPU steps gate everything) and the sphere-dominated phase at minute 30–40 (fragmentation of 6-core tasks; allocation 40.6).

6. Artifacts

Item	Path
Figures (PDF vector + PNG 300 dpi)	~/DeepNeuroBench/figures/exp10/
Slurm 24.11	slurm/slurm.conf (priority/basic, bf_interval=5, no PriorityTier, cpu-spill2/3), slurm/gres.conf.node{2,3}, slurm/deploy_shards.sh
Config / scripts	config/deepprep.slurm.exp10.config, stats/deepPrep_exp10.sh, stats/spill_controller2.sh
deepprep.nf	bold_pre_process, bold_transform_chain cpus 2 (backup deepprep.nf.orig-exp8)
Run outputs	/mydata/output/chess_data/exp10/QC/{report.html,timeline.html,trace.txt}
Telemetry	stats/telemetry/exp10/ (launch.log, monitor.log, spill_controller.log, dstat, per-core dstat, nvidia-smi dmon, power CSV)